LED Control via USB 2.0 D+ Pin Manipulation on Linux

Using libusb Test Modes and PCIe-Level Reset

Aarush Dilawri, Vivek Darji
Department of Electrical Engineering
Indian Institute of Technology Hyderabad
Supervised by Dr. GVV Sharma

***Abstract*—This paper presents a method to control an LED using a Linux laptop's USB 2.0 data pin (D+), without any microcontroller, serial adapter, or dedicated GPIO hardware. Two circuit variants are implemented and analyzed: one using a diode for current steering, and one using only resistors with a VBUS pull-up on D+. The software stack exploits USB 2.0 Test Modes via libusb to electrically manipulate the D+ pin, and uses a PCIe-level bus reset to exit the hardware silicon lock imposed by test mode. LED blinking is achieved by alternating between test mode and PCIe reset. An extension to a seven-segment display using a CD4015 shift register is also attempted and analyzed, with findings on the fundamental clock establishment limitation via the D- pin.**

***Index Terms*—USB 2.0, Test Mode, libusb, xHCI, PCIe reset, LED control, D+ pin, shift register, CD4015**

I. Introduction

Standard LED control from a computer requires a microcontroller with dedicated GPIO pins. A laptop's USB port does not expose GPIO pins, but the USB 2.0 data lines (D+ and D−) can be electrically manipulated using test modes defined in the USB 2.0 specification [1]. This project exploits these test modes to achieve full software-driven LED control from a Linux laptop using only a resistor, an LED, and optionally a diode.

A. Constraints

- No microcontroller, Arduino, or USB-to-serial adapter
- Full software control from Linux userspace
- Minimum external components

B. Challenges

USB 2.0 Test Modes are one-way hardware latches. The USB 2.0 specification, Section 7.1.20 [1], states that the only exit from test mode is a power-on reset. Additionally, the keyboard and mouse share the same xHCI controller (Bus 001), so entering test mode causes a bus panic that freezes all input devices.

II. USB Hardware Fundamentals

A. USB Port Pin Layout

A standard USB Type-A connector has four pins as defined in USB 2.0 Specification, Section 6.3 [1]:

TABLE I
USB Type-A Pin Assignments

Pin	Name	Description
1	VBUS	+5V power, always on, up to 500mA
2	D−	Differential data (negative)
3	D+	Differential data (positive)
4	GND	Ground reference

B. USB 2.0 Test Modes

USB 2.0 defines compliance test modes in Section 7.1.20 [1]. These modes force specific electrical states on D+ and D−:

TABLE II
USB 2.0 Test Mode Electrical States

Selector	Name	D+	D−
1	Test_J	3.3V	0V
2	Test_K	0V	3.3V
3	Test_SE0_NAK	0V	0V
4	Test_Packet	Test pattern	

Test_SE0_NAK (Selector 3) forces both D+ and D− to 0V (Single-Ended Zero), which is exploited to turn the LED off.

III. Circuit Design

A. Circuit 1: With Diode

Connections:

```
VBUS(5V) ---[300Ω]--- LED ---[Diode→]---
                        D+
                       GND
  ---------------------------------------
                       GND
```

Diode orientation: anode toward LED, cathode (stripe) toward D+.

Resistor calculation:

$$R = \frac{V_{VBUS} - V_{LED}}{I} = \frac{5\text{V} - 2\text{V}}{10\text{mA}} = 300\,\Omega \tag{1}$$

Operating principle:

When D+ is at its normal idle voltage (∼3.3V), the diode cathode is at higher potential than the anode, placing the diode in reverse bias. Current flows freely through the LED. When Test_SE0_NAK forces D+ to 0V, the diode becomes forward biased and steers all current away from the LED toward D+ (ground), turning the LED off.

TABLE III
DIODE CIRCUIT SWITCHING LOGIC

D+ Voltage	Diode State	LED
∼3.3V	Reverse biased	ON
0V	Forward biased	OFF

B. Circuit 2: Resistor Only (No Diode)

Connections:

```
VBUS(5V) ---[2.2kΩ]--- D+ --- LED --- GND
```

Operating principle:

The 2.2kΩ resistor feeds current from VBUS onto the D+ node. In normal state, current flows from VBUS through the resistor, through D+, through the LED, and to GND. When Test_SE0_NAK fires, the xHCI controller's output driver actively pulls D+ to 0V, overpowering the resistor pull-up. With D+ at 0V and the LED cathode at GND (also 0V), no potential difference exists across the LED and it turns off.

$$I_{normal} = \frac{V_{VBUS} - V_{LED}}{R} = \frac{5\text{V} - 2\text{V}}{2200\,\Omega} \approx 1.4\text{mA} \quad (2)$$

The resistor value of 2.2kΩ was determined experimentally. Values below 330Ω prevented the xHCI driver from pulling D+ to 0V; values above 1.8MΩ provided insufficient current to illuminate the LED.

TABLE IV
RESISTOR-ONLY CIRCUIT SWITCHING LOGIC

D+ State	Voltage	LED
Normal	∼VBUS via R	ON
Test_SE0_NAK	0V	OFF

IV. SOFTWARE IMPLEMENTATION

A. libusb and USB Control Transfers

libusb is an open-source C library providing userspace applications direct access to USB hardware, bypassing the OS driver stack [2]. Communication uses USB Control Transfers, defined in USB 2.0 Specification, Section 8.5.3 [1].

The 8-byte SETUP packet fields, as defined in USB 2.0 Specification, Table 9-2 [1]:

The `bmRequestType` value of `0x23` is a bitmask defined in USB 2.0 Specification, Table 9-2 [1]: bit 7 = 0 (host-to-device), bits 6:5 = 01 (class request), bits 4:0 = 00011 (port recipient).

The `wIndex` field format for PORT_TEST is specified in USB 2.0 Specification, Section 11.24.2.13 [1]: high byte contains the test mode selector, low byte contains the port number.

TABLE V
USB SETUP PACKET FIELDS

Field	Value	Meaning
bmRequestType	0x23	Host-to-device, class request, port recipient
bRequest	0x03	SET_FEATURE
wValue	21	PORT_TEST (Table 11-17 [1])
wIndex	(3<<8)|9	Test mode 3, port 9

$$\begin{aligned} \texttt{wIndex} &= (3 \ll 8) \mid 9 \\ &= \texttt{0000 0011 0000 1001} = 777_{10} \end{aligned} \quad (3)$$

B. led.c

Listing 1. led.c — Forces D+ to 0V via USB Test Mode

```c
#include <stdio.h>
#include <stdlib.h>
#include <libusb-1.0/libusb.h>

int main(int argc, char *argv[]) {
    if (argc != 2) return 1;

    libusb_init(NULL);

    // Open Bus 001 Root Hub
    // VID 0x1d6b = Linux Foundation
    // PID 0x0002 = USB 2.0 Hub
    libusb_device_handle *dev_handle =
        libusb_open_device_with_vid_pid(NULL, 0x1d6b, 0
            x0002);
    if (!dev_handle) return 1;

    int port_number = 9;

    // wIndex: high byte = test selector (3=SE0_NAK)
    //         low byte  = port number
    // Per USB 2.0 Spec Section 11.24.2.13
    int wIndex_test = (3 << 8) | port_number;

    // SET_FEATURE(PORT_TEST) - USB 2.0 Spec Table 11-17
    libusb_control_transfer(dev_handle, 0x23, 0x03, 21,
                            wIndex_test, NULL, 0, 1000);

    libusb_close(dev_handle);
    libusb_exit(NULL);
    return 0;
}
```

C. Why Standard Resets Fail

Once Test Mode is active, the xHCI PORTSC register (bits 30:28) is locked at the silicon level. As stated in USB 2.0 Specification, Section 7.1.20 [1], the only exit is a power-on reset. Standard approaches fail:

TABLE VI
RESET APPROACH COMPARISON

Approach	Layer	Result
libusb_reset_device	USB	Fails
sysfs unbind/rebind	Linux driver	Fails
PORT_RESET (Feature 4)	USB	Fails
PCIe remove + rescan	PCIe	**Works**

The PCIe remove/rescan performs a full hardware reset of the xHCI chip, equivalent to a power cycle, clearing the test mode latch.

D. gui.py

Listing 2. gui.py — Mouse-driven LED control interface

```python
import tkinter as tk
import os
import time
import subprocess
import threading

def led_off():
    # Triggers Test_SE0_NAK -> D+ = 0V -> LED OFF
    os.system("sudo ./led 0")

def led_on():
    # PCIe remove: resets xHCI silicon, clears test mode
    subprocess.Popen(
        ["sudo", "tee",
         "/sys/bus/pci/devices/0000:00:14.0/remove"],
        stdin=subprocess.PIPE).communicate(b"1")
    time.sleep(1)
    # PCIe rescan: controller rediscovered, D+ returns to normal
    subprocess.Popen(
        ["sudo", "tee", "/sys/bus/pci/rescan"],
        stdin=subprocess.PIPE).communicate(b"1")

blinking = False

def blink():
    global blinking
    while blinking:
        led_off()
        time.sleep(2)     # LED OFF for 2 seconds
        led_on()
        time.sleep(0.25) # LED ON for 0.25 seconds

def start_blink():
    global blinking
    blinking = True
    t = threading.Thread(target=blink)
    t.daemon = True
    t.start()

def stop_blink():
    global blinking
    blinking = False

# Cache sudo password before keyboard freezes
os.system("sudo dmesg -n 1")
os.system("sudo -v")

root = tk.Tk()
root.attributes('-topmost', True) # Stay visible when KB frozen
root.geometry("300x200")

tk.Button(root, text="LED OFF", command=led_off,
          bg="red", width=20, height=3).pack(pady=5)
tk.Button(root, text="LED ON", command=led_on,
          bg="green", width=20, height=3).pack(pady=5)
tk.Button(root, text="BLINK", command=start_blink,
          bg="yellow", width=20, height=3).pack(pady=5)
tk.Button(root, text="STOP BLINK", command=stop_blink,
          bg="gray", width=20, height=3).pack(pady=5)

root.mainloop()
```

V. LED Blinking

A. Implementation

Blinking is implemented by alternating between Test_SE0_NAK (LED OFF) and PCIe reset (LED ON) in a background thread, preventing GUI freezing. The `threading` module runs the blink loop independently of the main GUI event loop.

B. Timing Analysis

The blink cycle timing is:

$$T_{off} = T_{test_mode} + T_{sleep} \approx 0 + 2\text{s} = 2\text{s} \tag{4}$$

$$T_{on} = T_{PCIe_reset} + T_{sleep} \approx 1.5\text{s} + 0.25\text{s} = 1.75\text{s} \tag{5}$$

The PCIe reset introduces an inherent ∼1.5 second delay and a brief flicker during controller re-enumeration. This is a fundamental hardware limitation of the approach — the flicker is the physical controller powering down and back up.

The asymmetric timing (2s OFF, 0.25s ON) was chosen to make the ON state appear as a brief pulse against a longer OFF period, visually compensating for the PCIe reset flicker.

VI. Seven-Segment Display Extension (Attempted)

A. Motivation

An extension was attempted to display digits 0–9 on a single-digit common-anode seven-segment display using a CD4015 dual 4-bit static shift register [4], with the same USB port as the only interface.

B. Available Pin States

From one USB port, four electrical states are achievable via test modes:

TABLE VII
Available Pin States for Shift Register Clocking

Mode	D+	D−	Achievable Without Reset
Test_J	3.3V	0V	Yes
Test_K	0V	3.3V	Yes
Test_SE0_NAK	0V	0V	Yes
Normal	3.3V	3.3V	No (requires PCIe reset)

C. Proposed RC Delay Solution

To clock the CD4015 without the Normal (1,1) state, an RC delay circuit was proposed on the D+ (Data) line:

```
D+ ---[300Ω]--- CD4015 Pin 7
      ---[330μF]--- GND
```

The RC time constant:

$$\tau = RC = 300\,\Omega \times 330\,\mu\text{F} \approx 100\,\text{ms} \tag{6}$$

The capacitor holds the Data line HIGH for ∼100ms after D+ is pulled to 0V, allowing the Clock (D−) to rise and latch a ‘1’ into the shift register before the capacitor discharges.

D. Circuit Connections

- VBUS → CD4015 VDD (Pin 16)
- GND → CD4015 VSS (Pin 8)
- GND → CD4015 Reset A (Pin 6), Reset B (Pin 14)
- D− → Clock A (Pin 9), Clock B (Pin 1)
- D+ → 300Ω → Data A (Pin 7), 330μF to GND
- Q3A (Pin 2) → Data B (Pin 15) (cascade bridge)
- CD4015 outputs → 2.2kΩ → Seven-segment segments
- COM → VBUS (common anode)

E. Failure Analysis

The seven-segment display extension failed to produce correct output. Manual clock pulses via a jumper wire confirmed the CD4015 and display were wired correctly. The failure was traced to the inability to establish a reliable clock signal through D− using test mode switching.

The root cause is the **port reset delay**: switching between test modes via libusb control transfers does not produce a clean, instantaneous clock edge on D−. The xHCI controller introduces a non-deterministic delay during test mode transitions that violates the CD4015 setup and hold time requirements. Unlike a GPIO pin which can toggle in nanoseconds, each test mode transition involves a full USB control transfer round-trip through the xHCI driver stack, introducing latency that prevents reliable digital clocking.

Additionally, the Normal (1,1) state — required to shift a '1' without the RC hack — cannot be achieved without a PCIe reset, which takes ∼1.5 seconds and is unsuitable for real-time clocking.

F. Potential Future Work

A possible solution would be to use a USB port on a separate xHCI controller (if available), allowing PCIe resets without affecting the keyboard. Alternatively, a microcontroller could be used solely for clock generation while the USB D+ pin controls the data line.

VII. Results

- LED successfully controlled (ON/OFF) via USB D+ pin manipulation using Test_SE0_NAK
- Two circuit variants implemented: diode-based and resistor-only
- LED blinking achieved with 2s OFF / 0.25s ON asymmetric timing
- PCIe remove/rescan confirmed as the only viable method to exit test mode silicon lock
- Seven-segment display extension unsuccessful due to clock establishment failure via D− under test mode switching latency

VIII. Conclusion

This work demonstrates that USB 2.0 data pins can be repurposed as software-controlled electrical switches by exploiting compliance test modes defined in the USB 2.0 specification. The key insight is that when USB-level and driver-level resets fail due to a silicon hardware lock, a PCIe bus reset provides a path to restore normal operation. Two functional LED control circuits were implemented with no microcontroller. The seven-segment display extension revealed a fundamental limitation: test mode switching via libusb cannot generate clock signals fast enough to reliably drive a shift register, due to the non-deterministic latency of xHCI test mode transitions.

References

[1] USB Implementers Forum, *Universal Serial Bus Specification, Revision 2.0*, Apr. 2000. [Online]. Available: https://www.usb.org/document-library/usb-20-specification

[2] libusb contributors, *libusb-1.0 API Reference*. [Online]. Available: https://libusb.sourceforge.io/api-1.0/

[3] Intel Corporation, *eXtensible Host Controller Interface (xHCI) for Universal Serial Bus, Specification 1.2*, May 2019. [Online]. Available: https://www.intel.com/content/www/us/en/products/docs/io/universal-serial-bus/extensible-host-controler-interface-usb-xhci.html

[4] Texas Instruments, *CD4015B CMOS Dual 4-Stage Static Shift Register Datasheet*, 2003. [Online]. Available: https://www.ti.com/lit/ds/symlink/cd4015b.pdf